Music Technology Engineering Mastery

Course III - An asynchronous, project-driven engineering curriculum

Primary path: Build working musical products that musicians enjoy using.

Three pillars: Music software | Physical instruments | Audio systems and infrastructure

Course philosophy: *Learn. Retrieve. Predict. Build. Measure and listen. Review. Revisit.*

Trading Engineering and Oxide Systems Engineering are not prerequisites.

Contents

Mission 3

1. Philosophy and learning model 3

2. Curriculum roadmap 4

Stage 0 - Music technology and product anatomy 5

Stage 1 - Sound, music, mathematics, and digital audio 6

Stage 2 - Real-time audio programming 7

Stage 3 - DSP, synthesis, sampling, and effects 8

Stage 4 - Audio plugins and instrument applications 9

Stage 5 - DAW and DJ software engineering 10

Stage 6 - MIDI, controllers, and musical communication 11

Stage 7 - Embedded musical instruments 12

Stage 8 - Audio electronics and hardware design 13

Stage 9 - Audio interfaces, drivers, and professional audio 14

Stage 10 - Music platforms and cloud engineering 15

Stage 11 - Machine learning for musical applications 16

Stage 12 - Integrated flagship products and open source 17

Stage 13 - Music-technology interviewing and career preparation 19

3. Application gates: do not wait for the whole course 20

4. Product testing is part of the curriculum 20

5. Resource stack 21

6. Anti-rabbit-hole rules 21

7. Relationship to the quant and Oxide courses 21

8. Recommended first move 22

Mission

Become an exceptionally compelling engineer for companies that build musical instruments, synthesizers, DJ equipment, digital audio workstations (DAWs), audio plugins, professional audio systems, and music software.

This curriculum covers **all nine engineering disciplines** identified in the career survey:

1. Embedded firmware for music equipment
2. Audio infrastructure and drivers
3. DAW and DJ application engineering
4. Digital signal processing (DSP)
5. Audio plugin development
6. Electronics and musical hardware engineering
7. Music software backends and platforms
8. Music AI and audio analysis
9. Professional audio and live-sound systems

The aim is not merely to learn the technology, but to **build working musical products** that musicians enjoy using.

This course builds on existing experience in Go, C, backend systems, and embedded systems. A completed low-latency trading or Oxide systems curriculum would provide additional strength in C++, performance, concurrency, operating systems, and networking, but **neither is a prerequisite**. Reuse already-mastered material rather than repeating it.

1. Philosophy and learning model

The target is a broadly capable music-technology engineer with **demonstrable depth**, not someone who has superficially sampled every discipline.

Three pillars

Pillar	Scope
A. Music software	C++, real-time audio engines, DSP, synthesis, sampling, effects, plugins, DAWs, DJ software, desktop interfaces, music AI
B. Physical instruments	Embedded firmware, analog/digital electronics, MIDI, USB, controls, audio interfaces, sensors, PCBs, hardware/software integration
C. Audio systems and infrastructure	Drivers, operating-system audio, audio networking, synchronization, cloud services, collaboration, reliability, and product delivery

Required learning cycle

Each major topic should follow this cycle:

1. **Learn:** Read or watch one focused explanation.
2. **Retrieve:** Close the material and explain the idea from memory.
3. **Predict:** State what an experiment should demonstrate.
4. **Build:** Implement the smallest meaningful example.
5. **Measure and listen:** Test the result technically and musically.
6. **Review:** Explain failures, surprises, and corrections.
7. **Revisit:** Repeat or extend the work later without relying on notes.

Use retrieval practice, spaced repetition, interleaving, and repeated reconstruction. Avoid passive video/course consumption without implementation.

Progression rules

- **No fixed deadlines.** Each stage ends at an explicit mastery gate.
- **Do not wait for perfect theory.** Start producing sounds and building usable tools early.
- **Do not equate scope with excellence.** A small, excellent synth is better than an unfinished DAW clone.
- **Don't postpone job applications** until all nine disciplines are mastered.
- **Keep playing and making music.** Musical usefulness and tactile ergonomics are engineering requirements.

2. Curriculum roadmap

Stage	Topic	Primary deliverable
0	Music technology and product anatomy	Architecture breakdowns of three products
1	Sound, music, mathematics, and digital audio	Audio signal and analysis experiments
2	Real-time audio programming	`audio-engine` C++ playback/mixing engine
3	DSP, synthesis, sampling, and effects	`synth-core` reusable DSP library
4	Audio plugins and instrument applications	Polished VST3/AU software synthesizer
5	DAW and DJ software engineering	`decklab` two-deck DJ application
6	MIDI, controllers, and communication	Custom performance MIDI controller
7	Embedded musical instruments	Standalone hardware synth/groovebox
8	Audio electronics and hardware design	Custom, tested audio/control PCB
9	Audio drivers and professional audio	`audio-transport-lab` and latency investigation
10	Music platforms and cloud engineering	`sound-library` backend and synchronization
11	Machine learning for musical applications	`music-assistant` evaluated intelligent features
12	Integrated flagship product and open source	Finished public products and contributions
13	Music-tech job preparation	Portfolio, demonstrations, and role-specific interview readiness

Important: This order is a learning dependency map, not a rule that you must finish everything before applying. Audio fundamentals and real-time programming come early; interview practice, documentation, user testing, and open source run alongside the main program.

Stage 0 - Music technology and product anatomy

Objective

Understand how real music products are assembled from software, firmware, electronics, controls, audio processing, and user experience.

Study

Dissect the architecture of representative products:

- An Ableton-style DAW
- A Pioneer DJ-style deck
- A standalone groovebox
- A digital hardware synthesizer
- A MIDI controller
- An audio interface
- A software synth/plugin

For each, trace the **audio signal path**, the **control/data path**, and the **storage/preset path**. Identify which subsystems are application code, embedded firmware, DSP, device drivers, or electrical hardware.

Assignment

Produce **three product architecture breakdowns**, each containing:

- A subsystem diagram
- The path from physical/user input to audible output
- A description of hardware and software boundaries
- Real-time requirements and likely failure modes
- What you would prototype first if building a smaller version

Mastery gate

You can explain the major subsystems of each product and why its design demands particular combinations of firmware, DSP, application software, and electronics.

Stage 1 - Sound, music, mathematics, and digital audio

Objective

Develop an engineering understanding of how sound is represented, analyzed, generated, and manipulated.

Core topics

Physical and perceptual sound

- Waveforms, frequency, amplitude, phase, and timbre
- Harmonics and overtones
- Loudness and decibels
- Hearing and perceptual tradeoffs

Digital audio

- Sampling and the Nyquist principle
- Quantization, bit depth, and sample formats
- Aliasing and anti-aliasing
- PCM, encoding, and digital audio files
- Buffers and time-domain representation

Mathematics for DSP

- Sinusoids and complex numbers
- Discrete-time signals
- Convolution
- Fourier transform and FFT
- Frequency-domain representations
- Basic difference equations and filter behavior

Music fundamentals relevant to engineering

- Intervals, scales, and chords
- Tempo, meter, and rhythm
- Envelopes, oscillators, filters, and modulation
- Synthesis and sampling vocabulary

Experiments

1. Generate sine, square, sawtooth, and noise signals with code.
2. Build a waveform and spectrum viewer.
3. Demonstrate aliasing and quantization and listen to the artifacts.
4. Implement a simple low-pass filter.
5. Compare signals with the same fundamental frequency but different harmonic content.

Mastery gate

Given a common audible phenomenon, explain it in both time and frequency domains, recreate it in code, and test whether your explanation predicts the result.

Anti-rabbit-hole boundary: Learn derivations as needed to build and reason correctly. Do not defer implementation until you have completed an entire mathematics sequence.

Stage 2 - Real-time audio programming

Objective

Write reliable C++ software that meets audio callback deadlines under real operating-system load.

Core topics

- Audio blocks, buffers, sample rates, and channel layouts
- Callback execution and deadlines
- Audio thread versus UI/background threads
- Buffer underflow/overflow, clicks, dropouts, and jitter
- Priority inversion and thread coordination
- Allocation avoidance and bounded work on the critical path
- Lock-free or wait-free communication where justified
- Parameter smoothing and denormal behavior
- Core Audio and the architecture of ALSA, JACK/PipeWire, WASAPI, and ASIO
- Instrumentation for missed audio deadlines

Project - `audio-engine`

Build a C++ engine that can:

- Play and mix samples
- Route audio through processing nodes
- Change parameters while audio is running
- Handle start/stop and lifecycle changes
- Expose audio diagnostics
- Run under controlled artificial CPU/disk/UI load

Test callback behavior and missed deadlines, rather than merely measuring average throughput.

Mastery gate

Your engine plays reliably while unrelated system activity occurs. You can explain how its threading and buffering prevent glitches, and you can measure the remaining limits.

Stage 3 - DSP, synthesis, sampling, and effects

Objective

Design and implement the core sound-generation and sound-processing algorithms used in musical instruments.

Topic map

Synthesis	**Effects**	**Analysis**
Subtractive	EQ and filters	FFT and spectrograms
Wavetable	Delay and reverb	Envelope detection
FM	Chorus and flanger	Pitch detection
Additive	Distortion and saturation	Transient/onset detection
Sampling	Compression and limiting	Tempo and rhythm analysis

Additionally study:

- Polyphony and voice allocation
- Modulation matrices and parameter routing
- Envelope generation and interpolation
- Filter stability and frequency response
- Oversampling and antialiasing
- Numerical stability
- SIMD and real-time DSP optimization when justified

Project - `synth-core`

Create a reusable C++ synthesis/DSP library containing:

- Oscillators and noise sources
- ADSR envelopes
- Filters
- Modulation
- Polyphonic voice management
- Basic sample playback
- Delay and at least one other effect
- Numerical tests and audio examples

Listen critically to the results. A numerically valid algorithm is not necessarily a musically useful one.

Mastery gate

You can implement, test, profile, and explain common synthesis/effects algorithms. You can identify audible artifacts and propose defensible improvements.

Optional tool: Faust for experimenting with DSP descriptions and generating implementations.

Stage 4 - Audio plugins and instrument applications

Objective

Transform DSP code into a musician-facing product that works inside major DAWs.

Study

- Plugin lifecycles and host integration
- VST3 and Audio Units; CLAP later
- Host-to-plugin parameters and automation
- MIDI event handling
- Preset and project state persistence
- Real-time-safe communication between UI and audio processing
- C++ UI frameworks, especially JUCE
- Cross-platform builds, packaging, versioning, and regression testing
- Plugin validation and host compatibility

Flagship project A - Professional-quality software synthesizer

Build a complete, playable VST3/AU instrument using `synth-core`.

Required features:

- Oscillators, filters, envelopes, and modulation
- Polyphony and velocity response
- A coherent, responsive graphical UI
- Preset management
- Parameter automation
- Reliable save/reopen behavior in a DAW
- A stable, glitch-resistant audio path
- Installation instructions and a short product demonstration

Validate in real hosts; use tools such as pluginval.

Mastery gate

A musician can install the plugin, open it in Ableton, produce a useful sound, automate it, save the project, reopen it, and hear the same result reliably.

Application milestone: This is the first major readiness point for audio-plugin, software-instrument, and some audio-engine positions.

Primary resources: JUCE course and VST3 Developer Portal.

Stage 5 - DAW and DJ software engineering

Objective

Understand the architecture behind timeline-based creative tools and DJ applications.

Study

DAW/app architecture

- Audio routing graphs and buses
- Multitrack mixing and sample-accurate scheduling
- Timelines, sequencing, and playback state
- Non-destructive editing, undo/redo, and persistence
- Plugin hosting
- Waveform visualization and desktop UI architecture

DJ-specific systems

- Beat grids, BPM analysis, cue points, and looping
- Pitch shifting and time stretching
- Tempo sync and beat alignment
- Jog-wheel and physical-controller behavior
- DJ library metadata and browsing
- Real-time FX chains

Project - `decklab`

Build a compact two-deck DJ application with:

- Playback, cueing, looping, and mixing
- Tempo and pitch controls
- Waveform visualization
- Track library/metadata
- MIDI mapping
- A basic effects chain
- Basic beat synchronization

Test with physical DJ hardware when feasible. Study the architecture and contribution process of Mixxx.

Mastery gate

Perform a short real DJ mix with the application. Explain its playback architecture, synchronization behavior, and important limitations compared with commercial products.

Scope control: Do not attempt to clone every feature in Ableton Live or rekordbox.

Stage 6 - MIDI, controllers, and musical communication

Objective

Create expressive physical controls and reliable communication between musical devices and applications.

Study

- MIDI 1.0 notes, velocity, CC, pitch bend, clock, and SysEx
- MIDI over USB
- MIDI 2.0 and Universal MIDI Packets
- Device discovery and mappings
- Timing, synchronization, and jitter
- OSC
- BLE MIDI concepts
- Ableton Link and network synchronization
- Encoders, buttons, potentiometers, pads, and LED feedback
- Human factors and physical interaction design

Project - Custom performance MIDI controller

Build a hardware controller with physical controls, firmware, persistent mapping, and feedback.

Integrate it with your software synthesizer and DJ application. Consider a small display, LED rings, and remappable controls if they improve real musical use.

Mastery gate

The device works predictably during performance, persists mappings, communicates with software reliably, and can be used without detailed firmware knowledge.

Reference: MIDI Association and Ableton Link.

Stage 7 - Embedded musical instruments

Objective

Create a standalone digital instrument that makes sound without a host computer.

Study

- Embedded C/C++ audio architectures
- Microcontroller, DSP, and SoC tradeoffs
- Audio codecs, ADC/DAC, and audio output
- I^2S/SAI interfaces
- DMA and interrupt-driven sample processing
- Fixed-size audio buffers and real-time scheduling
- MCU RAM/flash limitations
- Preset storage, boot behavior, watchdogs, and firmware updates
- Portable DSP versus platform-specific optimizations

Use a practical audio development platform, such as Electrosmith Daisy or Bela.

Flagship project B - Standalone hardware instrument

Port suitable pieces of `synth-core` and implement:

- Audio generation or sample playback
- Physical inputs and an understandable UI
- Preset storage
- MIDI support
- Basic sequencing
- Reliable startup, recovery, and firmware updates
- Measurement of audio latency and resource use

The hardware can be a compact synth, sampler, or groovebox. Its precise form should follow what you most enjoy making music with.

Mastery gate

The instrument boots reliably, operates over extended sessions, responds expressively, preserves settings, and has a documented timing/resource profile.

Application milestone: With Stage 6 and some hands-on hardware debugging, this is a strong foundation for embedded musical-hardware firmware applications.

Stage 8 - Audio electronics and hardware design

Objective

Become capable of designing and debugging the analog/digital electronics surrounding audio software.

Study

- Voltage, current, impedance, and signal levels
- Op-amps and analog filter circuits
- Grounding, shielding, and noise
- ADC/DAC fundamentals
- Clocking and converter behavior
- Power regulation and analog/digital isolation concepts
- Connectors and protection
- PCB schematics, layout, manufacturing files, and bring-up
- Measurement with multimeters, oscilloscopes, and audio test signals
- Electromagnetic compatibility at a practical level

Project - Custom audio/control PCB

Use KiCad to design a low-voltage PCB for your instrument.

Start with a simple control/interface board, then increase complexity only when justified. Deliver:

- Schematic and PCB layout
- Bill of materials
- Assembly and bring-up instructions
- Test plan and test results
- Measured electrical behavior
- Diagnosis of at least one real fault or design revision

Mastery gate

You can read component datasheets, trace analog/digital signal paths, debug board-level issues, and bring up a manufactured PCB.

Safety: Begin with low-voltage, USB-powered systems. Avoid mains-powered circuit design without appropriate expertise and safety controls.

Career qualification: This gives meaningful hardware/product fluency; it does not substitute for deep EE experience in specialist analog/RF hardware design.

Stage 9 - Audio interfaces, drivers, and professional audio

Objective

Understand audio from software callback to operating system, interface hardware, and professional audio network.

Study

- OS audio I/O stacks and device enumeration
- USB Audio Class concepts
- Sample-rate conversion and clock-domain crossings
- Buffering, latency compensation, and device synchronization
- Drivers and firmware/host interfaces
- Multichannel routing and digital mixers
- Networked audio and audio-over-IP
- Packet timing, jitter buffers, dropped packets, and recovery
- AES67 concepts and professional network synchronization
- Redundancy, monitoring, and failure diagnosis

Project - `audio-transport-lab`

Stream low-latency multichannel audio between two computers.

Implement and analyze:

- Capture and playback
- Packetization and transport
- Jitter buffering and clock drift
- Packet loss/disconnection recovery
- Measurements of end-to-end latency and audio quality
- Diagnostic tools/logs for identifying glitches

Mastery gate

You can isolate and explain timing failures across application, driver, network, and device boundaries, with supporting measurements.

Stage 10 - Music platforms and cloud engineering

Objective

Apply existing backend and distributed-systems strengths to cloud capabilities used by music creators and connected devices.

Study

- Preset libraries and sample catalogs
- Search and metadata models
- Cloud storage and asset lifecycle
- User accounts, permissions, licensing, and entitlements
- Cross-device synchronization and conflict resolution
- Collaboration and sharing
- Device configuration and update services
- Event-driven workflows and observability
- Music-product UX implications for sync and offline operation

Project - `sound-library`

Build a backend for:

- Preset/sample metadata
- Search and user collections
- Synchronization between plugin, controller, and instrument
- Versioned presets and schema evolution
- Sharing/collaboration where useful
- Secure APIs and offline-recovery behavior

Use Go unless a different language provides a clear technical benefit.

Mastery gate

Changes propagate predictably across applications/devices, conflicts are resolved deliberately, and tests demonstrate recovery under failure.

Application milestone: Existing backend experience may already qualify you for music-platform positions. Do not postpone these applications for the DSP/hardware curriculum.

Stage 11 - Machine learning for musical applications

Objective

Implement, evaluate, and productize useful intelligent music features.

Study

- Onset, beat, tempo, and key estimation
- Pitch and harmonic analysis
- Timbre and audio embeddings
- Musical similarity and retrieval
- Source separation and transcription concepts
- Conventional DSP versus ML approaches
- Inference latency and resource limits
- Model evaluation and dataset quality
- Copyright, licensing, provenance, and responsible model use

Begin with classical DSP and audio analysis. Use Python for experimentation, and integrate mature libraries/models when appropriate rather than treating foundation-model training as a prerequisite.

Project - `music-assistant`

Implement **at least two** features, such as:

- BPM/key estimation for `decklab`
- Similar-sound sample search
- Section/transient detection
- Stem separation using an existing model
- Preset or instrument recommendations

Create reproducible evaluation data. Compare predicted versus measured behavior and include bad cases, latency, and limits.

Mastery gate

You can defend the algorithm, evaluation methodology, resource costs, and practical musical value of each feature.

Career qualification: Deep audio-ML research would require an additional advanced specialization; the present gate targets solid audio-ML software engineering.

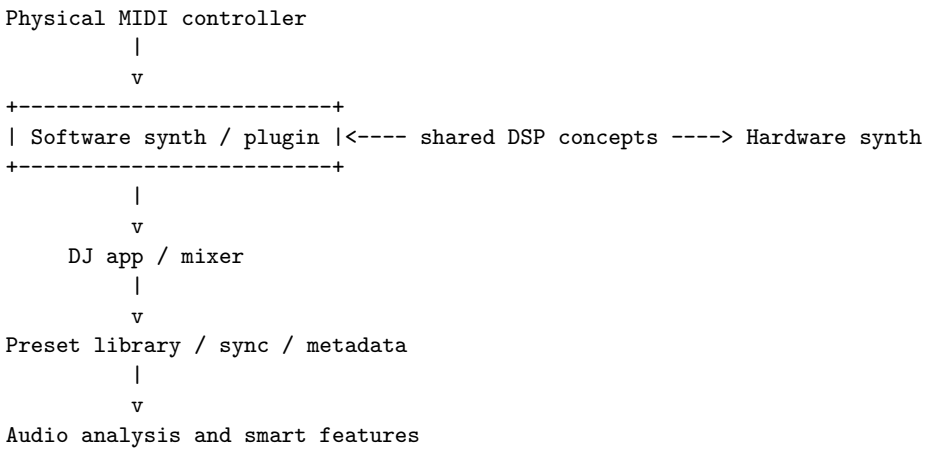
Resources: librosa tutorials and Essentia.

Stage 12 - Integrated flagship products and open source

Objective

Turn the course's projects into an unusually convincing public body of music-technology engineering work.

Preferred integration architecture



Keep components modular. They do not need to become one huge monolithic product.

Four flagship portfolio artifacts

1. A polished synthesizer plugin

Demonstrates C++, DSP, real-time programming, plugins, desktop UX, and musical judgment.

2. A standalone groovebox/synth

Demonstrates embedded firmware, audio interfaces, MIDI, electrical design, and physical interaction.

3. A DJ application with controller integration

Demonstrates real-time desktop software, playback architecture, synchronization, and creative workflows.

4. An audio-systems/music-platform laboratory

Demonstrates backend systems, device sync, audio networking, reliability, and music analysis.

Quality expectations

For each flagship artifact, publish:

- Working code and an understandable build/install procedure
- Automated tests and realistic manual tests
- Architecture diagrams and major design decisions
- Measurements where relevant
- A short video or audio demonstration
- Limitations and known failures
- User feedback and improvements

Open-source contribution track

Study and contribute to real music/audio codebases when useful:

- Mixxx
- Surge XT
- ChowDSP
- pluginval
- Essentia

Start by building a project, running tests, understanding a subsystem, reproducing a bug, and submitting a tested change. Respond to review thoughtfully.

Mastery gate: You have multiple polished public artifacts, a technical narrative you can defend, and ideally at least one substantive externally reviewed contribution.

Stage 13 - Music-technology interviewing and career preparation

Objective

Translate your technical and musical work into credible applications and realistic interview performance.

Interview preparation by role

Role	Emphasize
Audio DSP	Filters, DSP math, sampling, FFT, numerical stability, C++
Plugin/DAW engineer	C++, audio threading, plugin APIs, GUI/audio synchronization, architecture
Embedded firmware	C/C++, interrupts, DMA, I^2S, device debugging, deterministic timing
Hardware engineer	Circuit analysis, test equipment, PCB design, audio signal integrity
Professional audio	Drivers, audio I/O, networking, clocks, fault tolerance
Backend/platform	Distributed systems, API/data design, reliability, scalability
Music AI	Signal analysis, evaluation, inference, ML tradeoffs

Practice formats

- Timed coding exercises
- Short DSP implementation challenges
- Recorded design explanations
- Debugging sessions with deliberate faults
- Product walkthroughs
- Async code reviews from experienced engineers
- Occasional external mock interviews, without making scheduling a learning dependency

Portfolio demonstration standard

For every substantial project, prepare to show:

1. A concise demonstration of its musical use
2. The architecture
3. One difficult engineering problem
4. A bug or failure investigation
5. A measurable tradeoff or improvement
6. What you would change in a production version

Mastery gate

You can interview credibly for at least one chosen specialty and present compelling, technically defensible examples of your work.

3. Application gates: do not wait for the whole course

Target	Reasonable point to begin applying
Music backend/platform	Potentially now, with targeted domain preparation
Audio engine/DSP/plugin	After Stages 1-4, with a polished plugin
DAW/DJ application engineer	After Stages 1-5, with `decklab`
MIDI/controller firmware	After Stages 6-7, supported by real-time audio basics
Instrument firmware	After Stage 7 and hands-on hardware debugging
Hardware engineer	After Stage 8 and a functioning PCB; deeper EE may still be required
Professional audio systems	After Stage 9, with networking and diagnostics evidence
Music AI engineer	After Stage 11 and a reproducible evaluated ML project
Cross-disciplinary product engineer	After multiple finished, integrated portfolio projects

These are **learning milestones**, not guarantees that a particular employer will consider you qualified.

Three completion levels

Level 1 - Employable specialty candidate

A relevant functioning project, knowledge of the role's core engineering, and credible interview readiness.

Level 2 - Strong cross-disciplinary engineer

Meaningful software, embedded, DSP, electronics, and audio-system experience with well-documented public work.

Level 3 - Exceptionally compelling specialist with broad product fluency

One or more musically polished products, unusual depth in selected disciplines, external reviews/contributions, and ability to defend architectural choices throughout the audio stack.

Level 3 is the ambitious target. It does **not** mean being simultaneously equivalent to a lifelong analog-circuit specialist, senior DSP researcher, embedded architect, and principal cloud engineer.

4. Product testing is part of the curriculum

Music technology is not judged only by benchmarks or test suites. Continue DJing, producing music, and experimenting with instruments during the entire course.

For each major project:

- Use it in a real session, not only a scripted demo.
- Ask another musician to try it without guidance.
- Note confusing controls, poor ergonomics, sound-quality weaknesses, and workflow friction.
- Treat artistic and usability feedback as inputs to the engineering backlog.

A perfectly stable plugin that musicians find frustrating is not an exceptional product. A controller with excellent firmware and poor tactile usability is similarly incomplete.

5. Resource stack

Use a limited set of primary resources and add specialized materials only as they become necessary.

Subject	Starting resource
Audio plugin/real-time C++	JUCE course
DSP and synthesis experimentation	Faust documentation
Embedded audio	Bela tutorials
Hardware synth firmware	Daisy documentation
MIDI	MIDI Association
VST3	Steinberg Developer Portal
PCB design	KiCad resources
Music and audio analysis	librosa tutorials
Real DJ application source	Mixxx
Real synthesizer source	Surge XT
Plugin validation	pluginval
Audio-information retrieval	Essentia

6. Anti-rabbit-hole rules

1. **Don’t clone a commercial product.** Build a narrowly scoped instrument or creative tool that works well.
2. **Don’t separate theory from sound.** A math concept should lead quickly to audible implementation.
3. **Don’t design custom boards too early.** Prototype on desktop and inexpensive development hardware first.
4. **Don’t chase exotic DSP before the basics sound good.** Musical quality and robustness matter.
5. **Don’t reimplement standards and drivers unnecessarily.** Use established interfaces where possible.
6. **Don’t confuse technical success with product usefulness.** Incorporate musician feedback.
7. **Don’t let breadth block applications.** Apply when a specialty-specific gate is satisfied.
8. **Don’t collect repositories.** Polish the strongest products and make them easy to inspect.
9. **Don’t optimize without a measurement or audible reason.** Preserve correctness and sound quality.
10. **Don’t buy expensive hardware without a validated need.** Start with simulation and affordable development boards.

7. Relationship to the quant and Oxide courses

Capability	Quant trading	Oxide systems	Music technology
C++ systems programming	Essential	Useful	Essential
Performance engineering	Essential	Essential	Essential
Concurrency / real time	Essential	Essential	Essential
Operating systems	Important	Essential	Important
Networking	Important	Essential	Important
Embedded systems	Occasional	Important	Essential for physical products
DSP	Limited	Specialized	Essential
Electronics	Limited	Important	Essential for hardware roles
Cloud/distributed systems	Important	Essential	Useful
Technical product design	Useful	Important	Essential

These curricula share a strong systems-programming foundation. This third course adds a new domain: **the physics, mathematics, perception, and interaction design of sound and musical instruments**.

8. Recommended first move

Build a small but polished software synthesizer plugin first.

It is inexpensive to develop, immediately useful in Ableton, and introduces the key audio/DSP/real-time skills without buying or designing hardware. Later, porting the same synthesis concepts to a standalone embedded instrument creates a natural bridge to MIDI controllers, circuit design, and physical musical products.

The long-term goal is broad capability. The near-term goal is to **finish one excellent thing, learn from using it, and let that success finance the next layer of skill**.